Ex. No: 11	ADD iOS AS A BUILD TARGET
Date of Exercise	
Git-hub Host Link	

Aim:

To configure a Flutter application for the iOS platform and understand the process of building and testing the application using Xcode and an iOS Simulator.

Description:

Flutter supports cross-platform application development, including iOS. The iOS build target allows developers to configure, build, and test Flutter applications on iOS devices. The iOS platform files are generated using the `flutter create` command. The application can be opened in Xcode, tested using an iOS Simulator, and built for iOS distribution. Actual iOS building and testing require macOS and Xcode.

Procedure:

Step 1: Check Flutter Installation

Execute the following command to check the Flutter installation and development environment.

```
flutter doctor
```

Step 2: Check Available Devices

List the connected devices using:

```
flutter devices
```

Step 3: Create a Flutter Project

Create a new Flutter application named `ios_app`.

```
flutter create ios_app
cd ios_app
```

Step 4: Verify iOS Platform Files

Check the generated iOS project files.

```
dir ios
```

The project contains the `Runner`, `Runner.xcodeproj`, and `Runner.xcworkspace` directories.

Step 5: Analyze the Application

Check the application code for errors using:

```
flutter analyze
```

The analysis completed successfully with the message `No issues found!.`

Step 6: Open the iOS Project in Xcode

On a macOS system, open the iOS workspace using:

```
open ios/Runner.xcworkspace
```

Step 7: Select an iOS Simulator

Open Xcode, select an available iOS Simulator, and configure the application for execution.

Step 8: Run the Application

Execute the application using:

```
flutter run
```

Alternatively, run the application directly from Xcode.

Step 9: Test the Application

Verify the following application features:

- User interface
- Navigation
- Text fields
- Buttons
- Images
- Screen transitions

Step 10: Build the iOS Application

Build the application for iOS using Flutter on a supported macOS environment.

```
flutter build ios
```

Program:

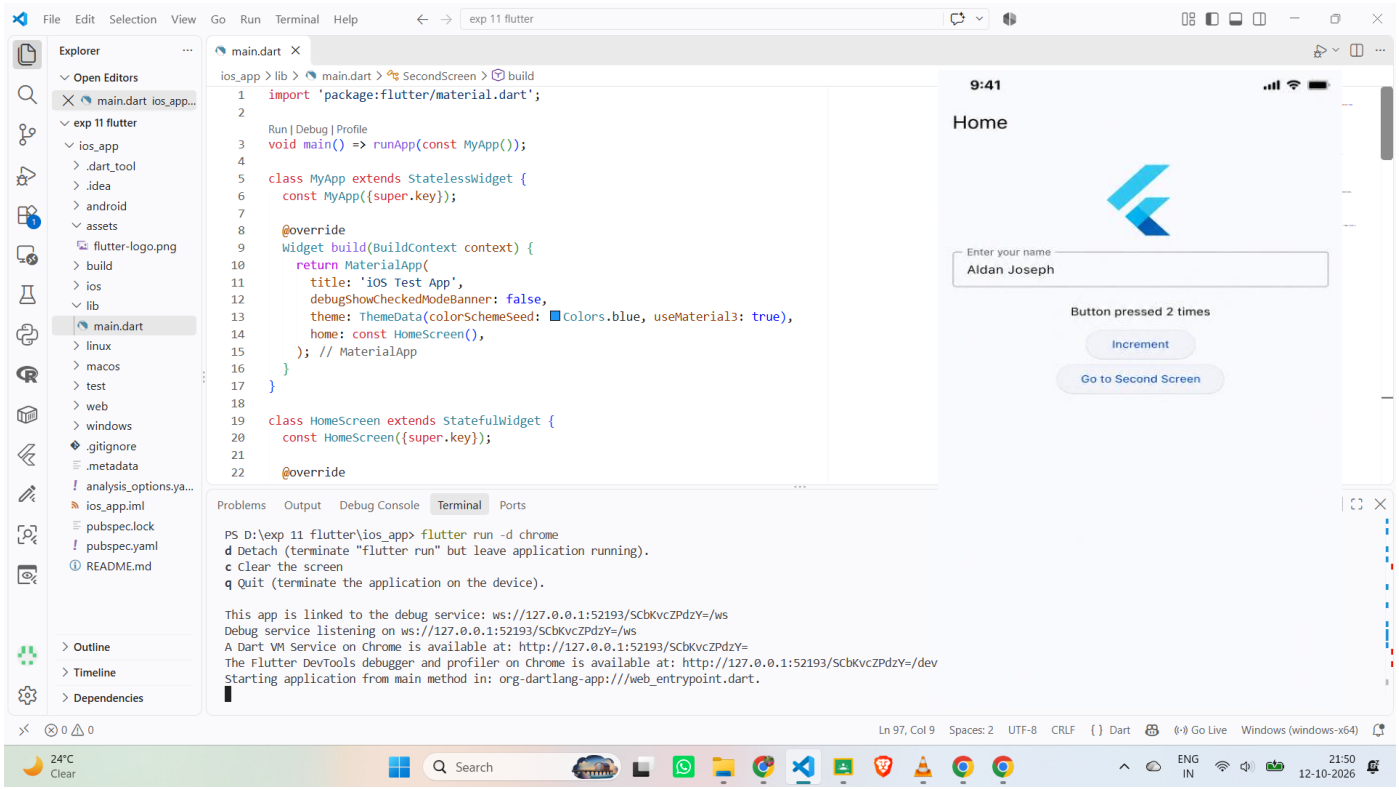
```
import 'package:flutter/material.dart';
void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'iOS Test App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const HomeScreen(),
    );
  }
}
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}
class _HomeScreenState extends State<HomeScreen> {
  final controller = TextEditingController();
  int count = 0;
  @override
```

```
void dispose() {
  controller.dispose();
  super.dispose();
}
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Home')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          Image.asset(
            'assets/flutter-logo.png',
            height: 80,
            errorBuilder: (context, error, stackTrace) =>
                const Text('Image not found'),
          ),
          const SizedBox(height: 16),
          TextField(
            controller: controller,
            decoration: const InputDecoration(
              labelText: 'Enter your name',
              border: OutlineInputBorder(),
            ),
```

```
        ),
        const SizedBox(height: 16),
        Text('Button pressed $count times'),
        ElevatedButton(
          onPressed: () => setState(() => count++),
          child: const Text('Increment'),
        ),
        ElevatedButton(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (_) => SecondScreen(name: controller.text),
              ),
            );
          },
          child: const Text('Go to Second Screen'),
        ),
      ],
    ),
  ),
 );
 }
}
```

```
class SecondScreen extends StatelessWidget {
  final String name;
  const SecondScreen({super.key, required this.name});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Second Screen')),
      body: Center(
        child: Text('Hello, ${name.trim().isEmpty ? "Guest" : name.trim()}!'),
      ),
    );
  }
}
```

Output:



RESULT

The Flutter project was created successfully, and the iOS platform configuration files were generated. Static code analysis was completed successfully without any issues. Actual iOS building and simulator testing require macOS and Xcode.